

KI-gestütztes Refactoring

Im Rahmen dieser Aufgabe analysieren und überarbeiten Sie die bereitgestellte C#-Applikation "ApocalypticFastFood", die in ihrer aktuellen Form grundlegende **SOLID-Design-Prinzipien verletzt**. Ihr Ziel ist die Entwicklung eines **systematischen, KI-basierten Refactoring-Konzepts**, das als wiederverwendbarer Leitfaden dient.

Sie erstellen ein **Markdown-Dokument, das Ihre Prompt-Entwicklung für KI-Systeme nachvollziehbar dokumentiert**. Dabei konzentrieren Sie sich auf die **methodische Konzeptionierung der Prompts selbst – nicht auf deren Outputs**. Beschreiben Sie präzise, wie Sie jeden Prompt entwickelt haben, welche Überlegungen dahinterstehen und wie die Prompts aufeinander aufbauen. Ihre Dokumentation soll zeigen, wie Sie **systematisch, effektiv und effizient** vorgegangen sind, um von der Analyse der SOLID-Verletzungen über das Refactoring bis zur Erstellung einer vollständigen Testsuite zu gelangen.

Das Konzept muss so gestaltet sein, dass die **sukzessive Anwendung Ihrer dokumentierten Prompts zu einer SOLID-konformen, wartbaren Codebasis** führt, die mit **xUnit und Reqnroll getestet** werden kann. Integrieren Sie dabei etablierte **C#-Namenskonventionen und Refactoring-Best-Practices**. Ihr Dokument dient als praktischer Leitfaden, den Sie oder andere Entwickler schrittweise anwenden können, um strukturierte Code-Transformationen durchzuführen.

Achten Sie auf eine **prägnante, gut verständliche und kompakte Darstellung**. Das fertige Konzeptdokument dokumentiert **Ihre Prompt-Engineering-Strategie, nicht die tatsächlich refaktorierten Code-Ergebnisse**. Die Abgabe erfolgt bis zum **23. November 2025**.

Bewertungsschema (max. 50 Punkte)

1. Systematische Prompt-Konzeptionierung (20 Punkte)

- **Nachvollziehbarkeit der Prompt-Entwicklung** (8 Punkte)
Klare Dokumentation des Entwicklungsprozesses jedes Prompts mit transparenter Begründung der Designentscheidungen
- **Methodische Strukturierung** (7 Punkte)
Logischer, aufeinander aufbauender Ablauf von der SOLID-Analyse über Refactoring bis zur Testsuite-Erstellung
- **Effizienz und Effektivität** (5 Punkte)
Optimierte Prompt-Strategie, die mit minimalem Aufwand maximale Wirkung erzielt

2. Fachliche Tiefe und SOLID-Prinzipien (15 Punkte)

- **SOLID-Prinzipien** (8 Punkte)
Vollständige Abdeckung aller fünf SOLID-Prinzipien mit konkreten Refactoring-Strategien
- **C#-Best-Practices** (4 Punkte)
Integration etablierter Namenskonventionen und Refactoring-Patterns
- **Testing-Konzept** (3 Punkte)
Durchdachte Integration von xUnit und Reqnroll in die Prompt-Strategie

3. Praktische Anwendbarkeit (10 Punkte)

- **Wiederverwendbarkeit** (5 Punkte)
Das Konzept ist als Leitfaden direkt anwendbar und auf andere Projekte übertragbar
- **Schrittweise Umsetzbarkeit** (5 Punkte)
Klare, sequenzielle Anwendungsschritte, die eigenständig durchführbar sind

4. Dokumentationsqualität (5 Punkte)

- **Prägnanz und Verständlichkeit** (3 Punkte)
Kompakte, klare Darstellung ohne unnötige Redundanzen
 - **Strukturierung** (2 Punkte)
Übersichtliche Gliederung mit sinnvoller Markdown-Formatierung
-

Prof. Dr. Carsten Müller | 26.10.2025